3

Unit 3: Register Transfer, Micro-operations and Basic Computer Design

Computer Organization and Architecture

CHANDRAS EDU

Never stop learning

Agenda

3.1 Register Transfer Language
3.2 Register Transfer
3.3 Bus & Memory Transfers
3.4 Arithmetic Micro-operations
3.5 Logic Micro-operations
3.6 Shift Micro-operations
3.7 Arithmetic Logic Shift Unit
3.8 Instruction Codes
3.9 Computer Registers
3.10 Computer Instructions
3.11 Timing and Control
3.12 Instruction Cycle
3.13 Memory Reference Instructions
3.14 Input-Output and Interrupt

3.1 Register Transfer Language

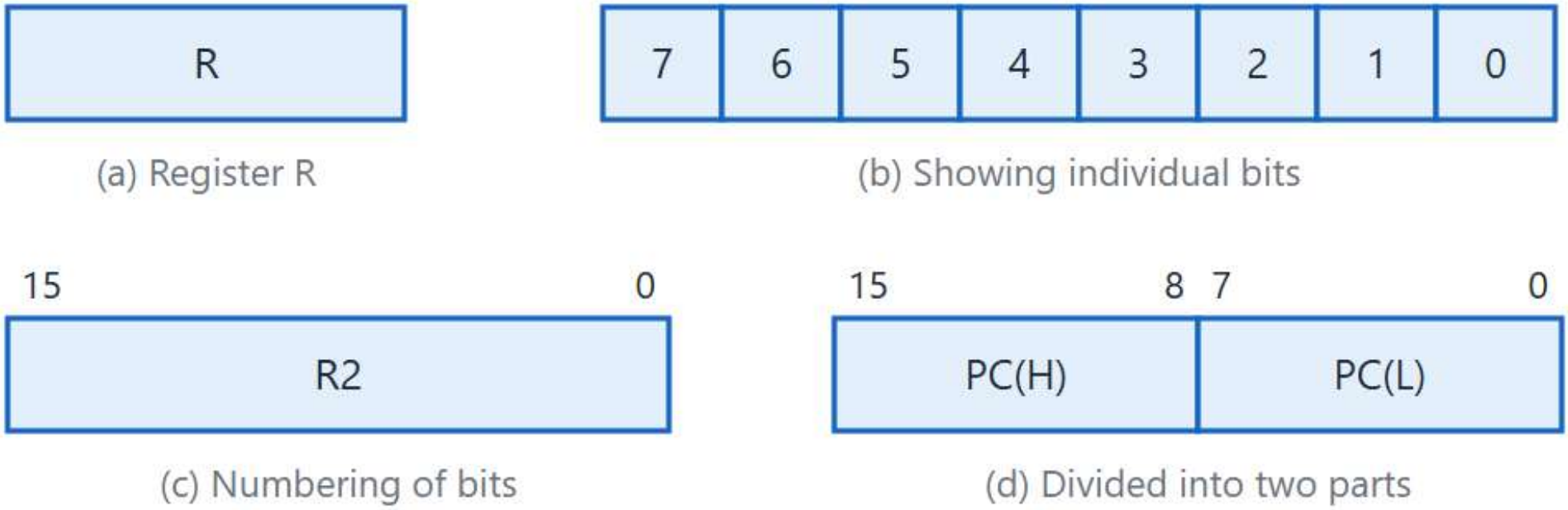
IN SIMPLE TERMS

Register transfer language is a short way to write what the hardware does each clock: R2 ← R1 means copy R1 into R2; P: R2 ← R1 means do it only when P = 1.

What RTL is

- A digital system is built from interacting modules: registers, decoders, arithmetic circuits
- **Register Transfer Language (RTL)** is a short notation for what moves where, and under what condition
- Registers are named with capital letters: **MAR**, **PC**, **IR**
- A register is drawn as a box, one cell per bit, most significant bit on the left
- Bits are addressed with brackets: **R1(0)** is one bit; **R1(7–0)** is a sub-field

Register block diagram



Block diagram of a register: plain box, individual bits, 16-bit numbering, and high/low byte partition. Adapted from M. Morris Mano, Computer System Architecture, Fig. 4-1.

The transfer statement

- **R2 ← R1** means: the content of R1 is transferred into R2
- The transfer is **nondestructive**: R1 keeps its content, only R2 is overwritten
- The arrow points to the **destination**
- It names real hardware: lines from R1's outputs to R2's inputs, plus a load-enable on R2

Control conditions and simultaneous transfers

Statement	Meaning
R2 ← R1	Copy R1 into R2
P: R2 ← R1	If P = 1, transfer R1 into R2 at the next clock edge
P: R2 ← R1, R1 ← R2	Both transfers at the same clock edge, using the values from before the edge (an exchange)

- P is a single bit from other logic; the colon separates **when** a transfer happens from **what** it does
- A comma separates statements that happen at the same clock edge

RTL symbols

Symbol	Meaning	Example
Capital letters	Register name	MAR, R2, IR
( )	A designated bit or range within a register	R1(0–7)
←	Transfer of information	R2 ← R1
:	Control condition: if true, do this on the next clock	P: R2 ← R1
,	Separates simultaneous transfers	R2 ← R1, R1 ← R2

3.2 Register Transfer

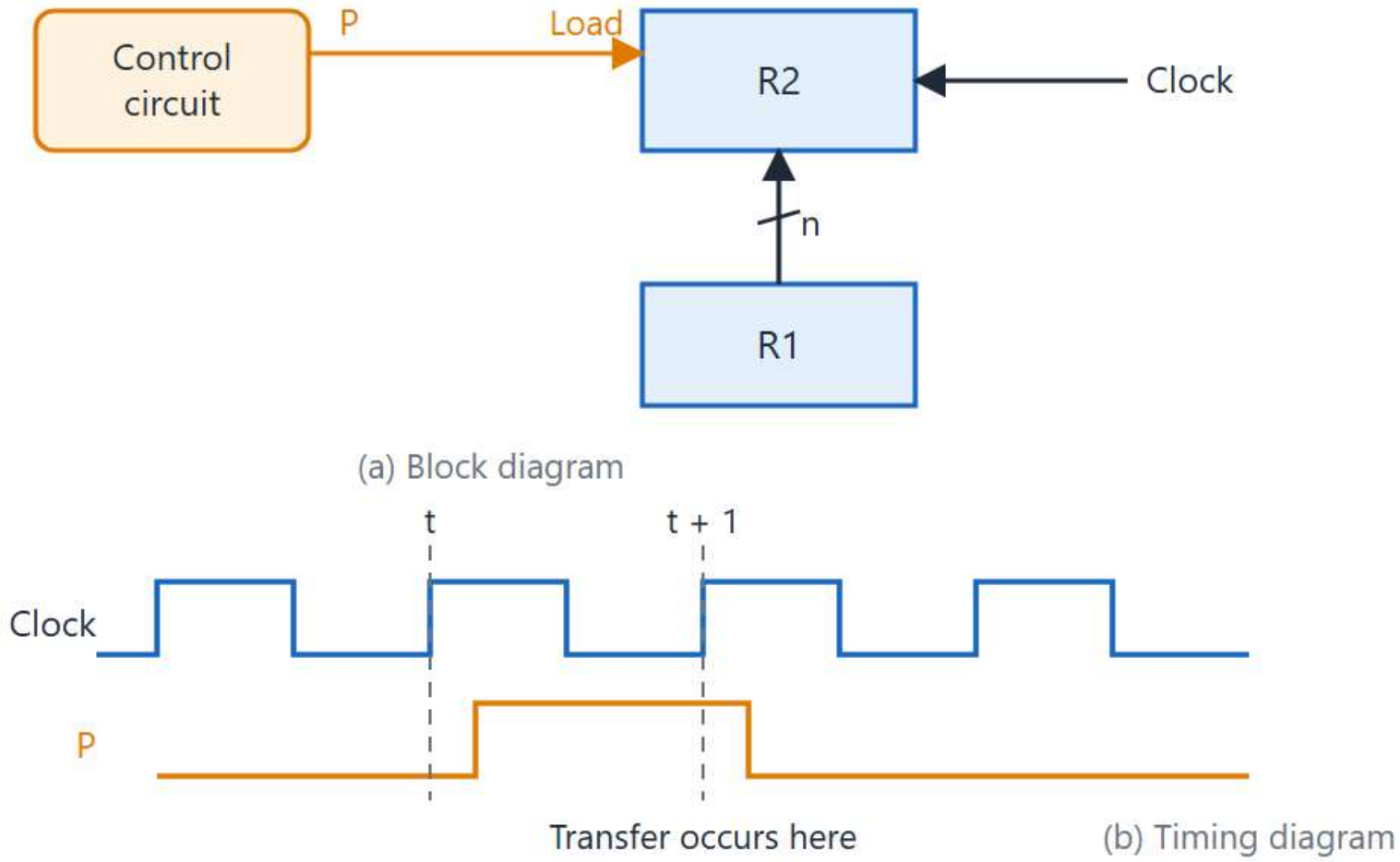
IN SIMPLE TERMS

R1’s outputs are wired to R2’s inputs all the time; the load signal decides whether R2 actually takes the value at the next clock edge.

What the hardware needs

- R1’s output lines are wired straight into R2’s input lines, always
- R2 has a **load-enable** input: it lets R2 take the value only when P = 1
- Both registers use the same clock, so transfers happen at the same instants
- Only R2’s load is controlled by P; R1 keeps its content whatever P does

Transfer from R1 to R2



Transfer from R1 to R2 when P = 1: (a) block diagram, (b) timing diagram. Adapted from M. Morris Mano, Computer System Architecture, Fig. 4-2.

Why not just gate the clock?

Gate the clock (avoid)

- Clock AND P is sent to R2
- P can change at any moment, not in step with the clock
- The gated clock can glitch: a tiny false pulse or an edge at the wrong time
- R2 may then load at the wrong time

Use load-enable (correct)

- The clock goes straight to R2, untouched
- P only tells R2 whether to load at the clock edge
- The clock never glitches
- R2 loads only when P = 1 at a clock edge

Timing

- P must be steady a short time **before** the clock edge (setup time)
- If P = 1 at a clock edge, R2 loads R1 at that edge
- In the figure, P goes high after edge t and is still 1 at edge t + 1, so R2 loads at t + 1
- R2 then keeps the new value until the next load

3.3 Bus & Memory Transfers

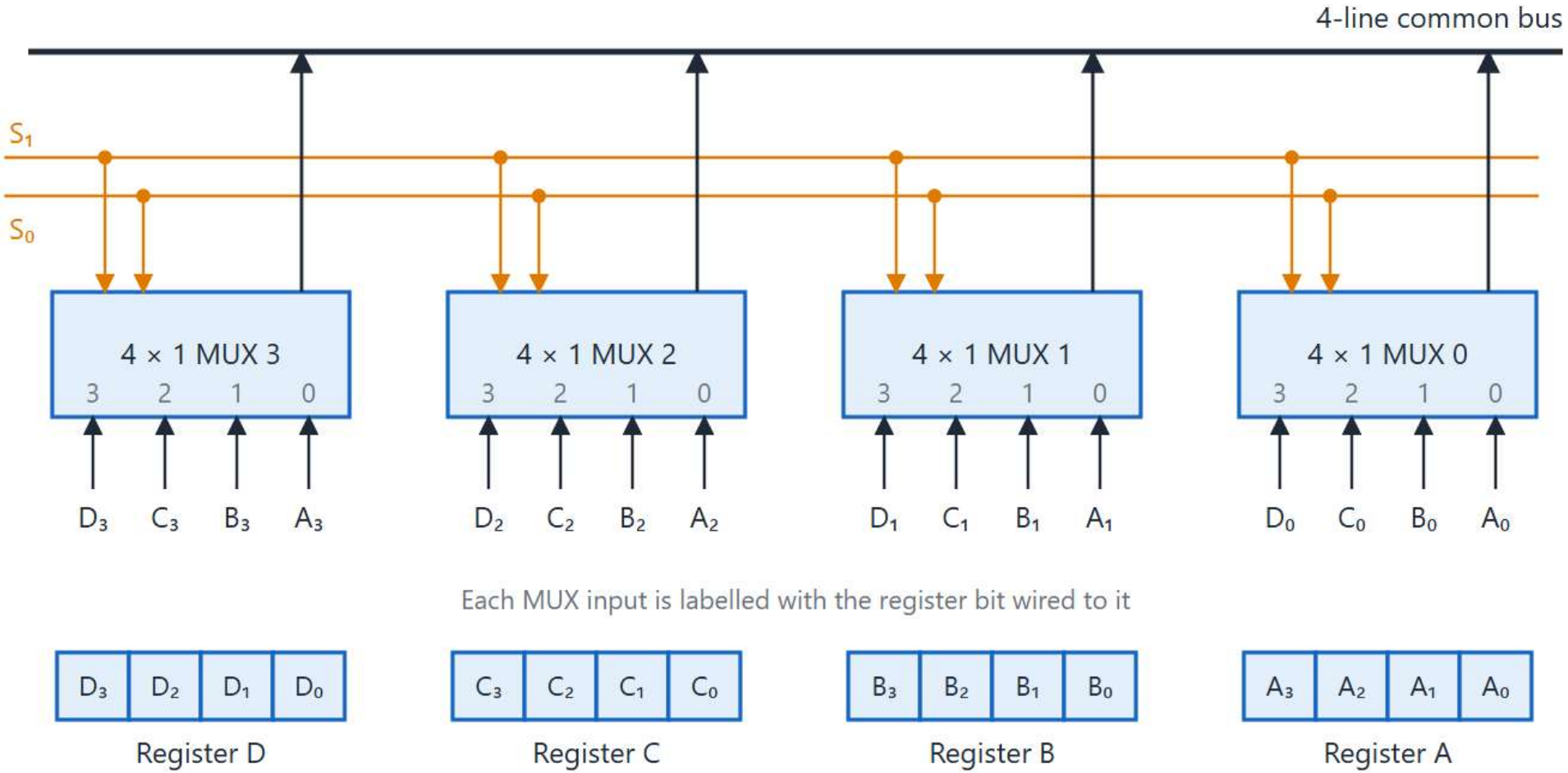
IN SIMPLE TERMS

Instead of wiring every register to every other, registers share one bus. The select lines decide which register’s value is on the bus at any moment.

The wiring problem

- Wiring every register to every other grows as roughly n^2 lines
- Fix: one shared set of lines, the **common bus**; only one register drives it at a time
- With multiplexers: one MUX per bit, with that bit from every register as its inputs
- Four registers need a 4-to-1 MUX per bit, all sharing the same 2-bit select code

Bus system for four registers



Bus system for four registers using multiplexers. S_1S_0 select lines connect to all four 4×1 MUXes; each drives one line of the 4-line common bus. Adapted from M. Morris Mano, Computer System Architecture, Fig. 4-3.

Using the bus

S_1S_0 = 10 (select C): BUS ← C, R1 ← BUS

- One MUX handles one bit; an n-bit bus needs n identical MUXes
- One 2-bit code chooses a register for the whole word
- Routing is combinational: no clock involved
- Only R1's load needs a clock edge, so both steps fit in one clock period

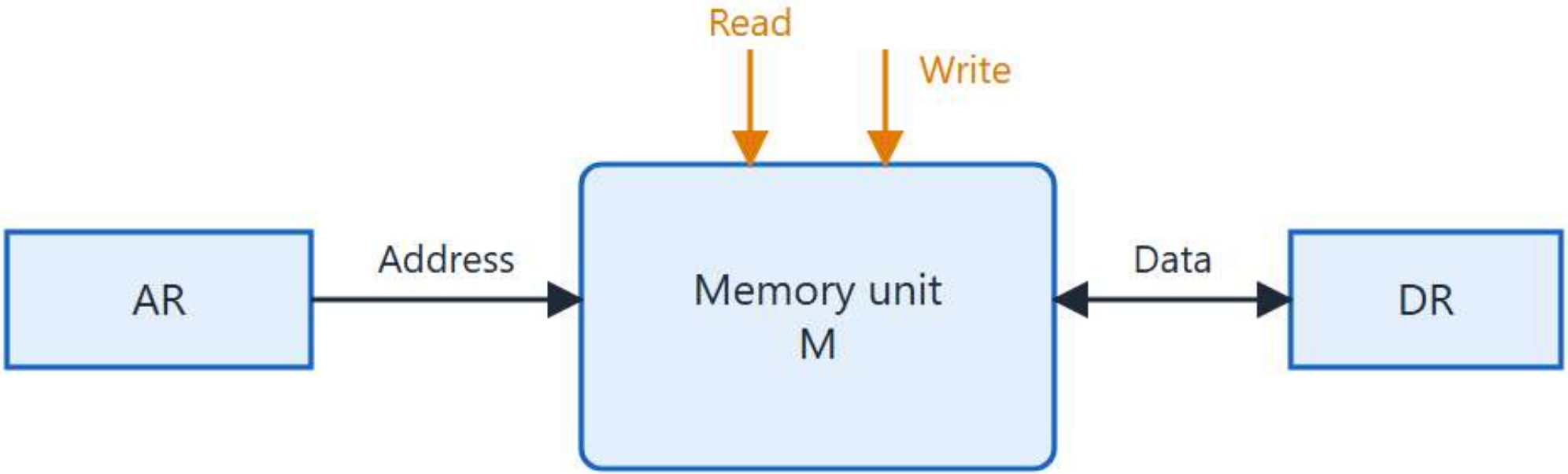
The three-state alternative

- Real buses often use **three-state buffers** instead of a MUX per bit
- Each buffer either drives 0 or 1, or is disconnected (high impedance, Z)
- Enable exactly one register's buffers at a time; the others do not interfere
- This is why datasheets describe a pin as "TRI-STATE"

Memory transfers

- An **address register (AR)** selects a memory word
- A **data register (DR)** holds the value being read or written
- Read and Write control lines tell the memory what to do

Memory with AR and DR



Memory connected to address register AR (supplying the address) and data register DR (receiving read data), with Read and Write control lines.

Memory RTL


```
Read:    DR ← M[AR]
Write:   M[AR] ← DR
```

- **M[AR]** means the memory word addressed by the current content of AR
- The brackets name a location that depends on AR's content, not a fixed register

3.4 Arithmetic Micro-operations

IN SIMPLE TERMS

One parallel adder does all the arithmetic micro-operations (add, subtract, increment, decrement) by changing what is fed into its second input and its carry.

What is a micro-operation?

- The smallest unit of work a CPU does in one clock cycle on data held in registers
- One register transfer, possibly with a simple operation along the way
- Three families in this unit: arithmetic, logic and shift
- Together they are exactly what an ALU is built from

Basic arithmetic micro-operations

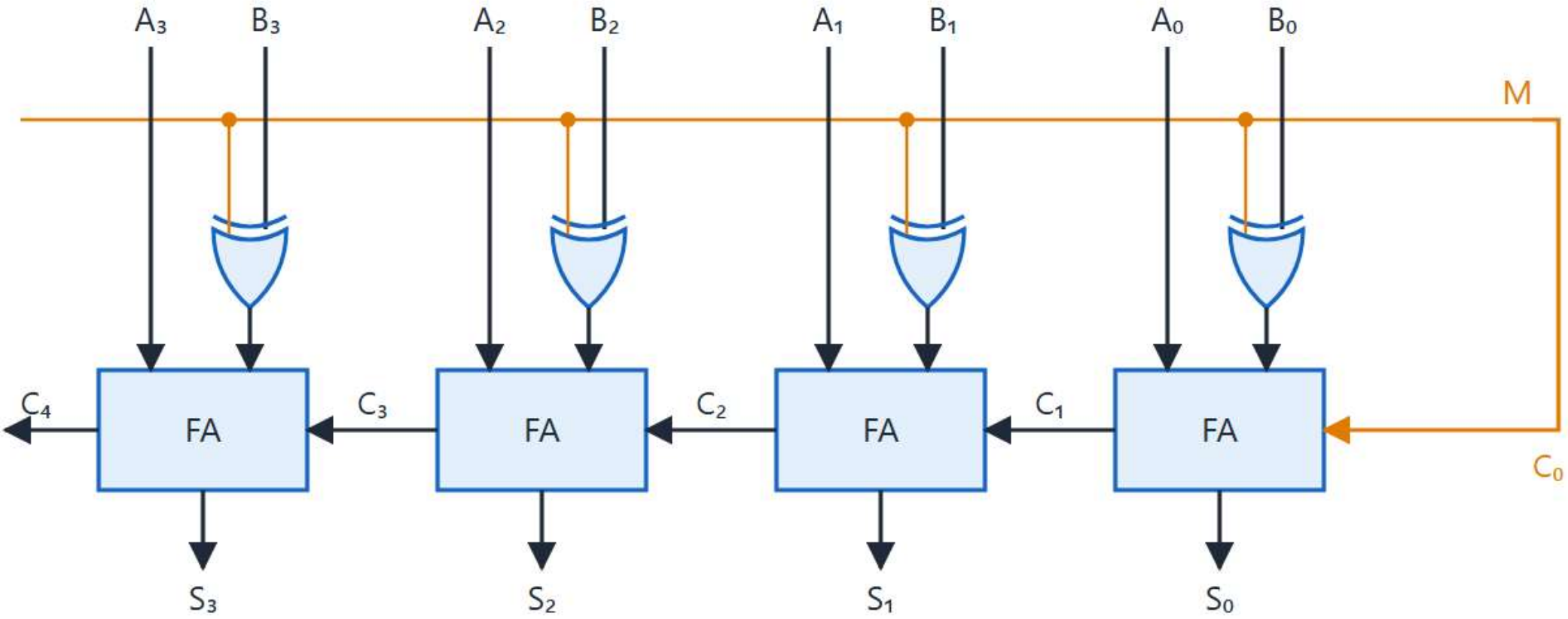
Operation	RTL statement
Addition	R3 ← R1 + R2
Subtraction	R3 ← R1 − R2 (implemented as R3 ← R1 + R2′ + 1)
Increment	R1 ← R1 + 1
Decrement	R1 ← R1 − 1

- Subtraction adds the 2's complement: R2′ (bitwise complement) plus a carry-in of 1

One circuit, four operations

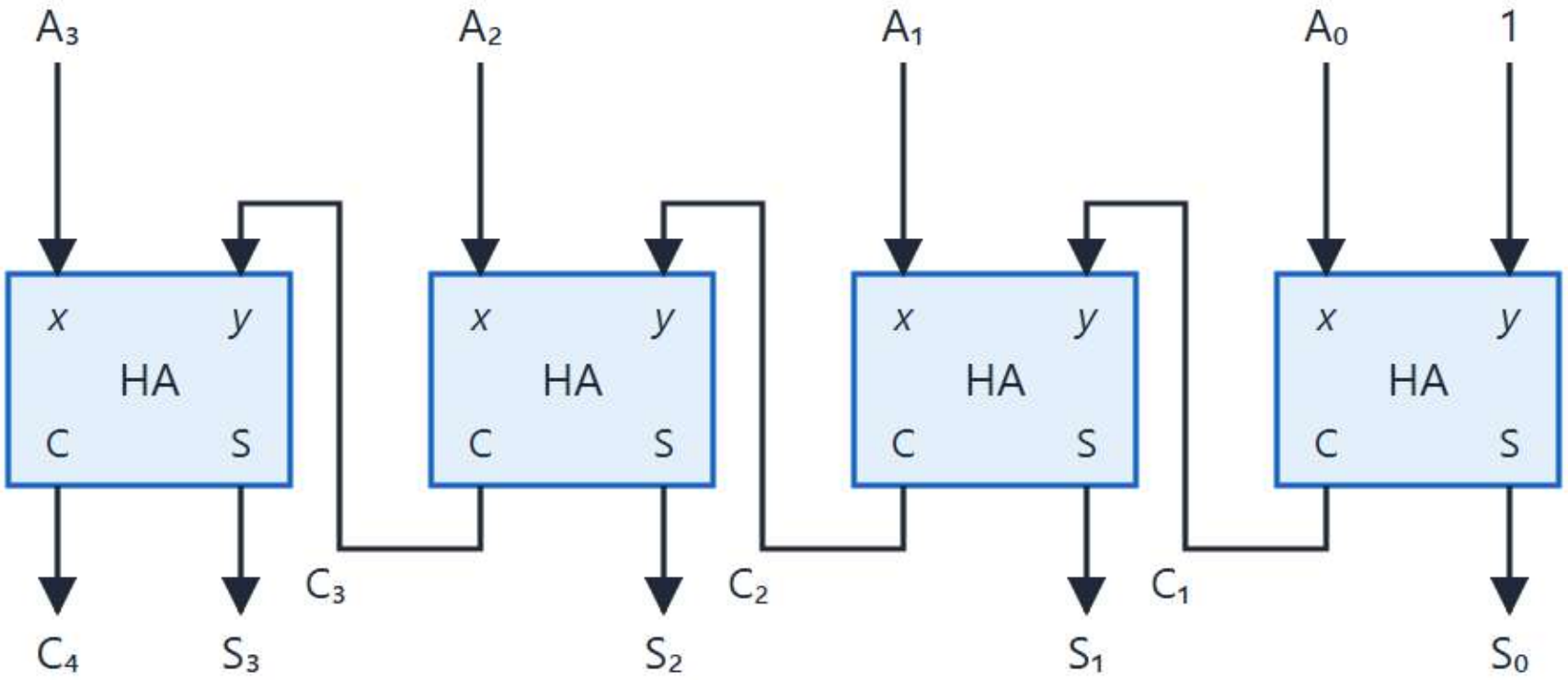
- All four run on a single n-bit **adder-subtractor**: a chain of full adders
- A mode control M is XORed into the B input and feeds the first carry-in
- M = 0 adds; M = 1 subtracts
- Increment: one operand hardwired to all 0s and the carry-in forced to 1

n-bit adder-subtractor



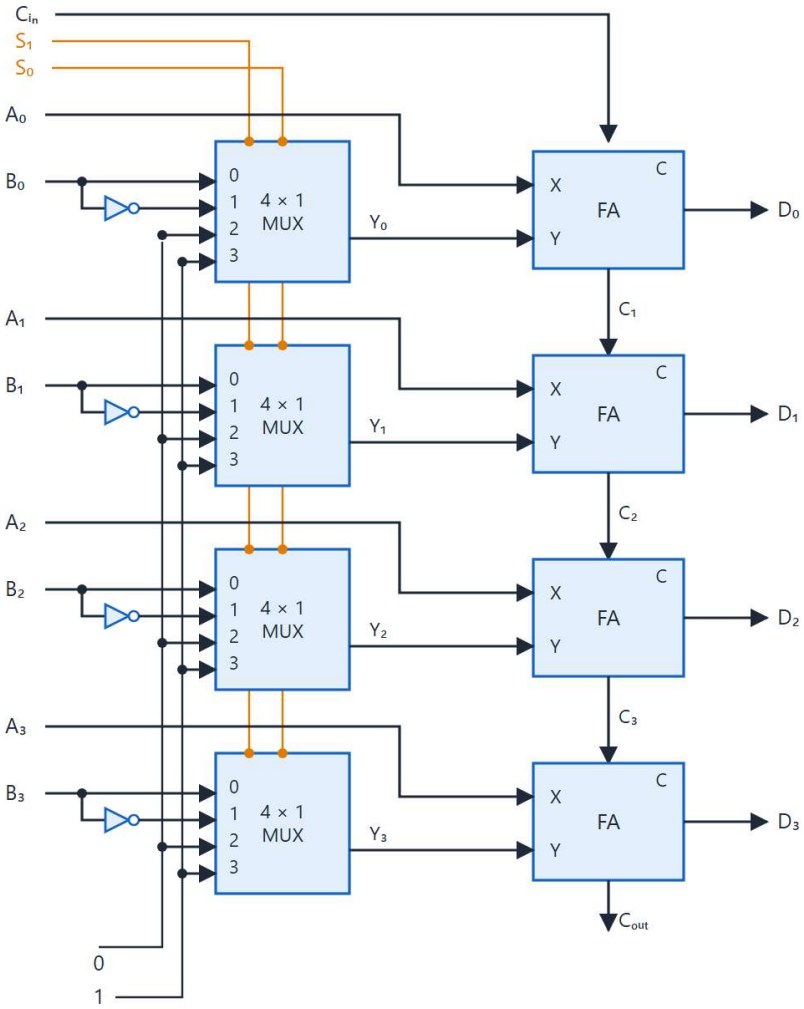
n-bit adder-subtractor: mode input M selects addition (M=0) or subtraction (M=1) via XOR gates and carry-in. Adapted from M. Morris Mano, Computer System Architecture, Fig. 4-7.

4-bit binary incrementer



4-bit binary incrementer. Adapted from M. Morris Mano, Computer System Architecture, Fig. 4-8.

4-bit arithmetic circuit



- One parallel adder; its Y inputs come from 4 × 1 multiplexers
- S_1S_0 choose B, B′, all 0s or all 1s for Y
- The input carry C_{in} adds 0 or 1
- Output: $D = A + Y + C_{in}$

4-bit arithmetic circuit. Adapted from M. Morris Mano, Computer System Architecture, Fig. 4-9.

Arithmetic circuit function table

S_1	S_0	C_{in}	Y	$D = A + Y + C_{in}$	Micro-operation
0	0	0	B	D = A + B	Add
0	0	1	B	D = A + B + 1	Add with carry
0	1	0	B′	D = A + B′	Subtract with borrow
0	1	1	B′	D = A + B′ + 1	Subtract
1	0	0	0	D = A	Transfer A
1	0	1	0	D = A + 1	Increment A
1	1	0	1	D = A − 1	Decrement A
1	1	1	1	D = A	Transfer A

Arithmetic circuit function table. Adapted from M. Morris Mano, Computer System Architecture, Table 4-4.

Overflow detection

- Adding two **signed** numbers can give a result outside the representable range
- The adder itself still “succeeds”; that is **overflow**
- Compare the carry into the sign bit with the carry out of it
- $\mathbf{V = C_n \oplus C_{n-1}}$
- C_n is the final carry-out; C_{n-1} is the carry into the sign bit
- If the two disagree, the sign bit was corrupted: overflow

Worked example: overflow


```
  0101   (+5)
+ 0100   (+4)
  ----
  1001   reads as −7, but 5 + 4 = 9 does not fit
```

- 4-bit signed numbers, range −8 to +7
- C_4 (carry out of bit 3) = 0 and C_3 (carry into bit 3) = 1
- **$V = C_4 \oplus C_3 = 0 \oplus 1 = 1$**: overflow correctly flagged

3.5 Logic Micro-operations

IN SIMPLE TERMS

Logic micro-operations work on each bit separately, with no carries. With a mask in a second register you can set, clear or flip chosen bits and leave the rest alone.

Logic versus arithmetic

- Arithmetic treats a register as one number, with carries between bit positions
- Logic treats it as an array of **independent bits**: no interaction between neighbours
- That independence makes it ideal for changing individual bits on purpose

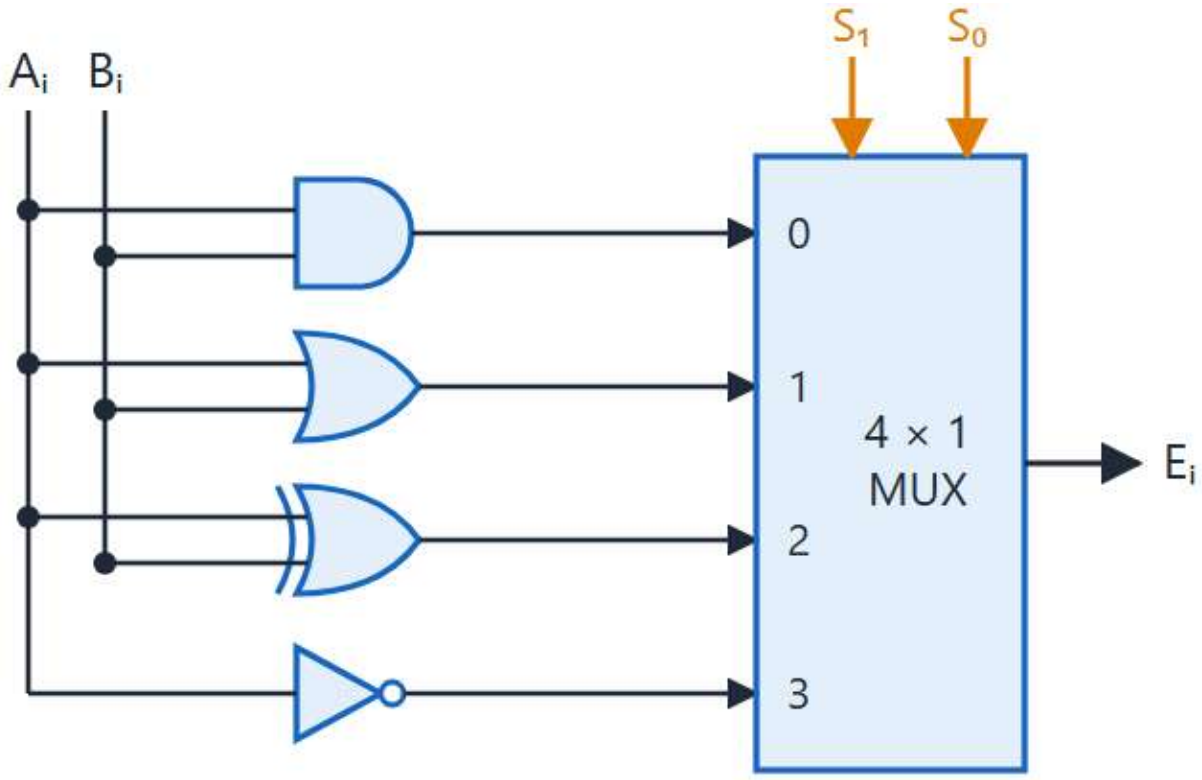
The four basic logic micro-operations

Operation	RTL statement	Per-bit rule
AND	$A \leftarrow A \wedge B$	1 only where both bits are 1
OR	$A \leftarrow A \vee B$	1 where either bit is 1
XOR	$A \leftarrow A \oplus B$	1 where the bits differ
Complement	$A \leftarrow A'$	every bit flipped

Hardware: one stage

- Each logic micro-operation needs only one gate per bit
- One stage has an AND, an OR, an XOR gate and an inverter
- A 4 × 1 multiplexer with select inputs S_1S_0 passes one result to E_i
- Repeat the stage n times for an n-bit logic circuit

One stage of the logic circuit



(a) Logic diagram

S_1	S_0	Output	Operation
0	0	E = A ∧ B	AND
0	1	E = A ∨ B	OR
1	0	E = A ⊕ B	XOR
1	1	E = A′	Compl.

(b) Function table

One stage of logic circuit: (a) logic diagram, (b) function table. Adapted from M. Morris Mano, Computer System Architecture, Fig. 4-10.

Selective bit manipulation with a mask

B is the mask: a 1 means “touch this bit of A”, a 0 means “leave it alone”.

Name	RTL	Effect on marked bits (B = 1)
Selective-set	$A \leftarrow A \vee B$	Forced to 1
Selective-complement	$A \leftarrow A \oplus B$	Flipped
Selective-clear	$A \leftarrow A \wedge B'$	Forced to 0
Mask (AND)	$A \leftarrow A \wedge B$	Kept as is; unmarked bits (B = 0) are forced to 0

Worked example: A = 1010 1100, B = 0000 1111

	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
A	1	0	1	0	1	1	0	0
B (mask)	0	0	0	0	1	1	1	1
Selective-set (A∨B)	1	0	1	0	1	1	1	1
Sel.-complement (A⊕B)	1	0	1	0	0	0	1	1
Selective-clear (A∧B′)	1	0	1	0	0	0	0	0
Mask (A∧B)	0	0	0	0	1	1	0	0

- Only b_3–b_0 (highlighted, where B = 1) are affected by set, complement and clear
- Mask keeps the marked bits but forces the unmarked ones to 0

Insert, and a free clear


```
A ← A ∧ 1111 0000    (clear the low nibble)
A ← A ∨ 0000 1010    (insert the new value 1010)
```

- **Insert** is two micro-operations: clear the target field with a mask, then OR in the new field value
- **A ← A ⊕ A** clears A to all 0s without a separate constant-0 source

3.6 Shift Micro-operations

IN SIMPLE TERMS

A shift moves every bit one place left or right. The three kinds differ only in what fills the empty end: a 0 (logical), the bit that fell off (circular), or a copy of the sign bit (arithmetic right).

Three kinds of shift

- A **shift** moves every bit of a register one position left or right in one clock cycle
- The direction of movement is the same in all three kinds
- They differ only in the value that fills the vacated bit
- **Logical**: a 0 fills it
- **Circular**: the bit that fell off fills it
- **Arithmetic right**: a copy of the sign bit fills it

Logical shift right (shr)

	b_3	b_2	b_1	b_0
Before (R = 1100)	1	1	0	0
After shr	0	1	1	0

- A 0 always fills the vacated position; the bit that falls off is discarded
- The rightmost bit (0) is lost and a fresh 0 (highlighted) enters at b_3
- Right choice for unsigned data with no sign bit to protect

Circular shift right (cir)

	b_3	b_2	b_1	b_0
Before (R = 1011)	1	0	1	1
After cir	1	1	0	1

- The bit that falls off is fed back in at the other end
- b_0's original value (1) reappears at b_3 (highlighted); nothing is discarded
- n circular shifts in the same direction return the register to its original value

Arithmetic shift

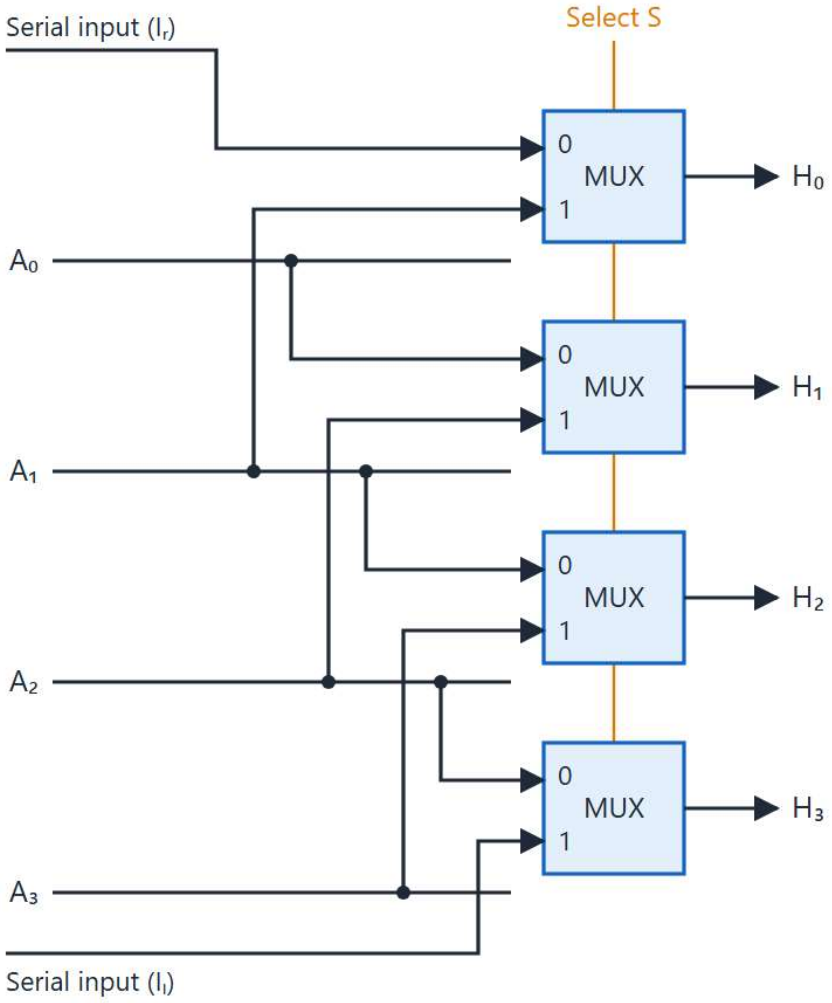
	b_3 (sign)	b_2	b_1	b_0
Before (R = 1001, i.e. −7)	1	0	0	1
After ashr	1	1	0	0

- Arithmetic **right** shift fills the vacated bit with a copy of the sign bit
- 1100_2 = −4, and −7 ÷ 2 rounded toward −∞ is −4; a logical shift would turn the number positive
- Arithmetic **left** shift moves bits exactly like a logical left shift (0 fills the bottom)
- It means “multiply by 2”, so check overflow: if the sign bit changes, the result no longer fits

A combinational shifter

- Built from four 2-input multiplexers sharing one select line S
- S = 0: each output H_i takes the bit above it, so the data shifts right; serial input I_R enters H_0
- S = 1: each H_i takes the bit below it, so the data shifts left; I_L enters H_3
- One gate delay, no clock edge; the result is loaded into a register at the next clock
- No shift-register wiring needed; this is what feeds a CPU's ALU

4-bit combinational shifter



Function table

S	H_0	H_1	H_2	H_3
0	I_r	A_0	A_1	A_2
1	A_1	A_2	A_3	I_l

S = 0: shift right (down)
S = 1: shift left (up)

- Four 2-input MUXes share one select line S
- S = 0: shift right; serial input I_R enters H_0
- S = 1: shift left; I_L enters H_3
- One gate delay, no clock edge

4-bit combinational circuit shifter. Adapted from M. Morris Mano, Computer System Architecture, Fig. 4-12.

3.7 Arithmetic Logic Shift Unit

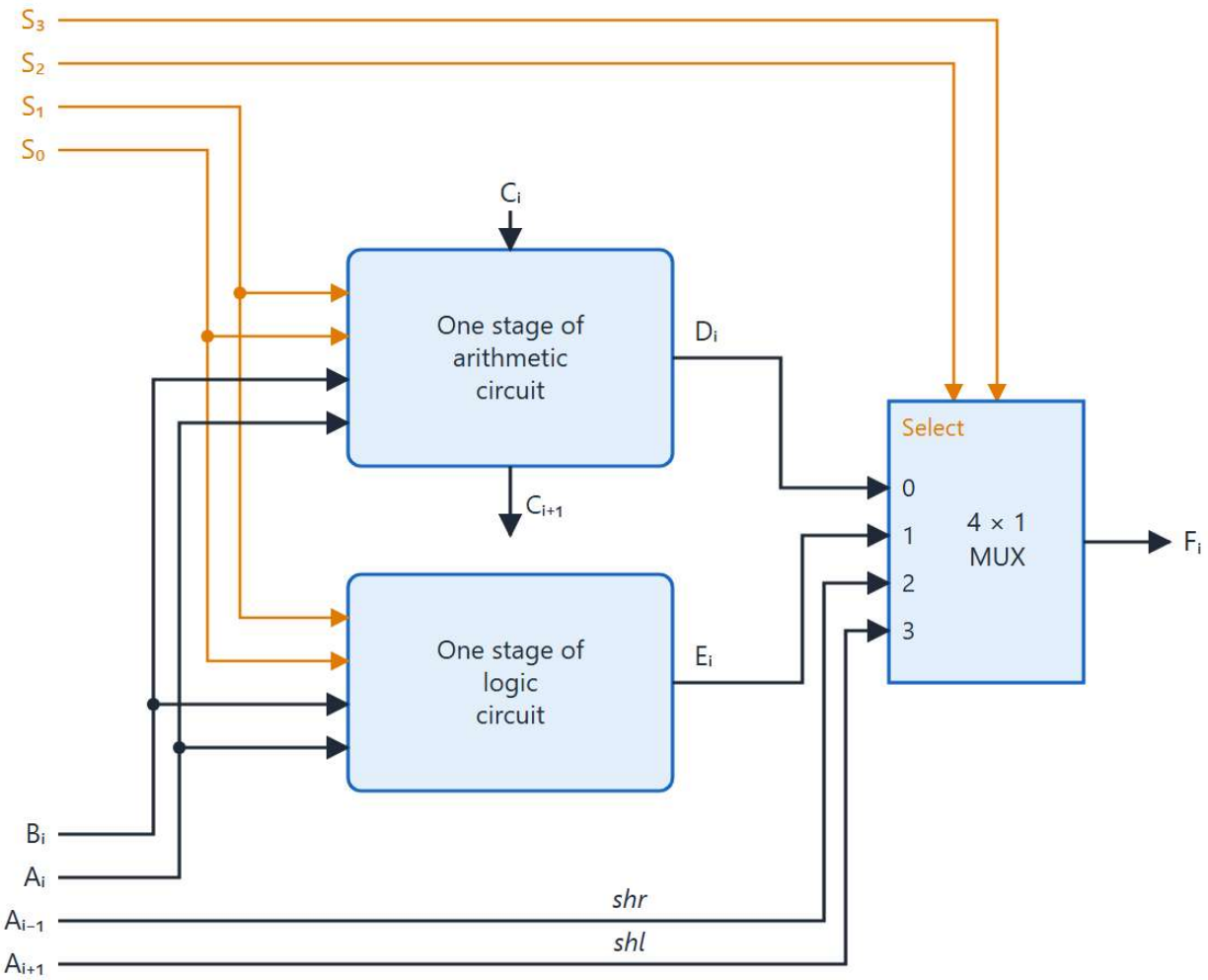
IN SIMPLE TERMS

The ALU computes arithmetic, logic and shift results side by side; the select lines decide which one becomes the output.

The structure

- One combined block does any of the three families: the **Arithmetic Logic Shift Unit (ALU)**
- An arithmetic circuit, a logic circuit and a shifter are fed the same operands
- All three compute every cycle, in parallel
- S_3S_2 select, through a 4 × 1 multiplexer, which result reaches F
- It selects an entire n-bit result, not a single bit

One stage of the ALU



- All circuits compute every cycle, in parallel
- S_3S_2 select, through a 4 × 1 MUX, which result reaches F
- It selects an entire n-bit result, not one bit
- Five selection variables give 14 operations

One stage of arithmetic logic shift unit. Adapted from M. Morris Mano, Computer System Architecture, Fig. 4-13.

The select code

- Five selection variables, S_3, S_2, S_1, S_0 and C_{in}, give **14 operations**
- S_3S_2 = 00 selects an arithmetic operation
- S_3S_2 = 01 selects a logic operation
- S_3S_2 = 10 or 11 selects a shift
- Every row is a micro-operation from the last three topics; the ALU adds only the selection logic

ALU function table: arithmetic

S_3	S_2	S_1	S_0	C_{in}	Operation	Function
0	0	0	0	0	F = A	Transfer A
0	0	0	0	1	F = A + 1	Increment A
0	0	0	1	0	F = A + B	Addition
0	0	0	1	1	F = A + B + 1	Add with carry
0	0	1	0	0	F = A + B′	Subtract with borrow
0	0	1	0	1	F = A + B′ + 1	Subtraction
0	0	1	1	0	F = A − 1	Decrement A
0	0	1	1	1	F = A	Transfer A

Function table for arithmetic logic shift unit. Adapted from M. Morris Mano, Computer System Architecture, Table 4-8.

ALU function table: logic and shift

S_3	S_2	S_1	S_0	C_{in}	Operation	Function
0	1	0	0	×	F = A ∧ B	AND
0	1	0	1	×	F = A ∨ B	OR
0	1	1	0	×	F = A ⊕ B	XOR
0	1	1	1	×	F = A′	Complement A
1	0	×	×	×	F = shr A	Shift right A into F
1	1	×	×	×	F = shl A	Shift left A into F

- × means don’t care

Function table for arithmetic logic shift unit. Adapted from M. Morris Mano, Computer System Architecture, Table 4-8.

3.8 Instruction Codes

IN SIMPLE TERMS

An instruction tells the computer what to do (opcode) and where the data is (address). The I bit says whether the address is the data's location itself or the location of its address.

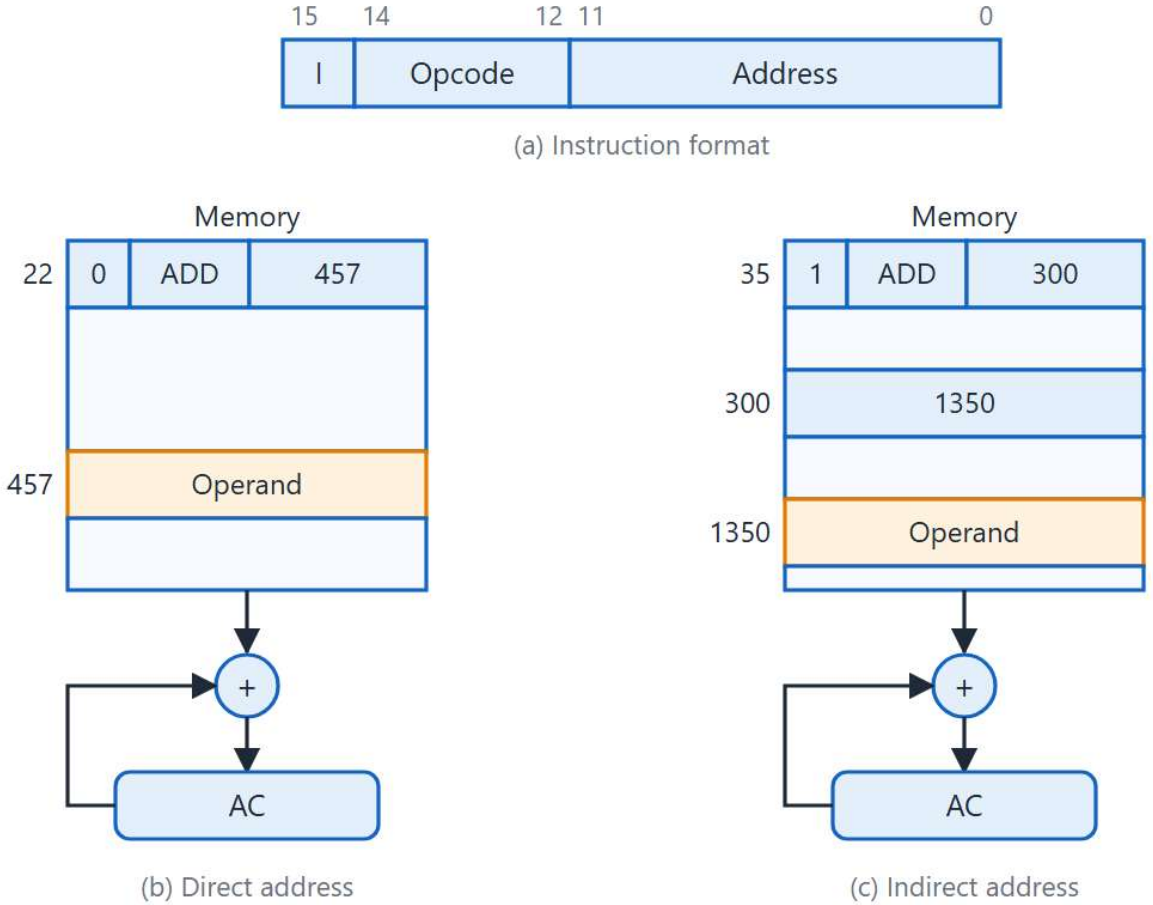
The Basic Computer

- From here to the end of the unit we build one concrete machine: the **Basic Computer**
- Memory: 4096 words of 16 bits each, so a 12-bit address
- Stored program: the same memory holds data and instructions
- A word is an instruction when the CPU treats it as one, not because of its bit pattern

The 16-bit instruction word

- Every instruction is one 16-bit word with three fields: **I** (bit 15), **opcode** (bits 14–12), **address** (bits 11–0)
- The 3-bit opcode distinguishes 8 categories
- 7 are **memory-reference** instructions, each with a 12-bit address: exactly enough for 4096 words
- The 8th pattern, **111**, has no address: its 12 bits are individually decoded
- That frees two more categories, register-reference and I/O, without using address space

Direct and indirect address



- (b) Direct: address field 457 is the operand's address
- (c) Indirect: field 300 points to a word holding 1350, the effective address
- Indirect costs one extra memory read

Instruction format with direct and indirect address. Adapted from M. Morris Mano, Computer System Architecture, Fig. 5-2.

Direct versus indirect addressing

I	Mode	Meaning
0	Direct	The address field is the effective address: AR ← IR(0–11)
1	Indirect	The address field points to a word that holds the effective address: AR ← M[IR(0–11)], one extra memory read

- Indirect trades one extra memory access for an effective address that can change at runtime: array indexing, pointer following

Recognising the instruction category

Opcode (IR bits 14–12)	I	Category
not 111	0 or 1	Memory-reference (bits 11–0 are the address)
111	0	Register-reference (bits 11–0 individually decoded)
111	1	Input-output (bits 11–0 individually decoded)

- Indirect addressing (I = 1) only means something for a memory-reference instruction
- For opcode 111 the same bit switches between register-reference and I/O

3.9 Computer Registers

IN SIMPLE TERMS

The Basic Computer has a handful of registers and one memory, all sharing a single 16-bit bus. Three select lines choose who talks on the bus; a load signal chooses who listens.

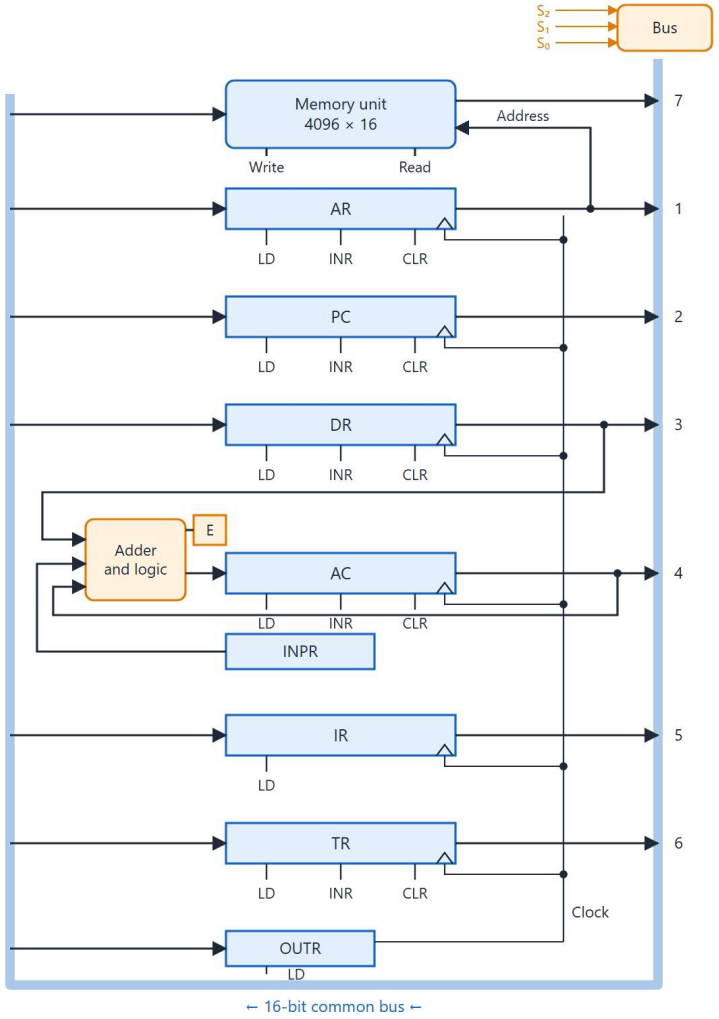
The register set

Register	Bits	Function
DR	16	Data Register: the memory operand being worked with
AR	12	Address Register: the memory address for the current access
AC	16	Accumulator: the general-purpose register most operations use
IR	16	Instruction Register: the instruction being executed
PC	12	Program Counter: address of the next instruction
TR	16	Temporary Register: intermediate values
INPR	8	Input Register: one character from an input device
OUTR	8	Output Register: one character for an output device
SC	4	Sequence Counter: times the control unit; not programmer-visible

Control flip-flops

Flip-flop	Meaning
I	Indirect-addressing bit of the current instruction
S	Start/stop: clearing it halts the clock
E	Extended AC bit: carry-out of addition, extra bit in a circular shift
R	Interrupt flip-flop: an interrupt is pending and about to be serviced
IEN	Interrupt Enable: master switch for interrupts
FGI	Input flag: a new character is ready in INPR
FGO	Output flag: the output device can accept a character in OUTR

Registers on one common bus



- $S_2S_1S_0$ pick the bus source: 1 = AR, 2 = PC, 3 = DR, 4 = AC, 5 = IR, 6 = TR, 7 = memory
- AR alone supplies the memory address
- AC loads from the adder and logic circuit, not the bus; the carry goes to E
- LD, INR, CLR load, increment and clear; IR and OUTR can only be loaded

Basic computer registers connected to a common bus. Adapted from M. Morris Mano, Computer System Architecture, Fig. 5-4.

3.10 Computer Instructions

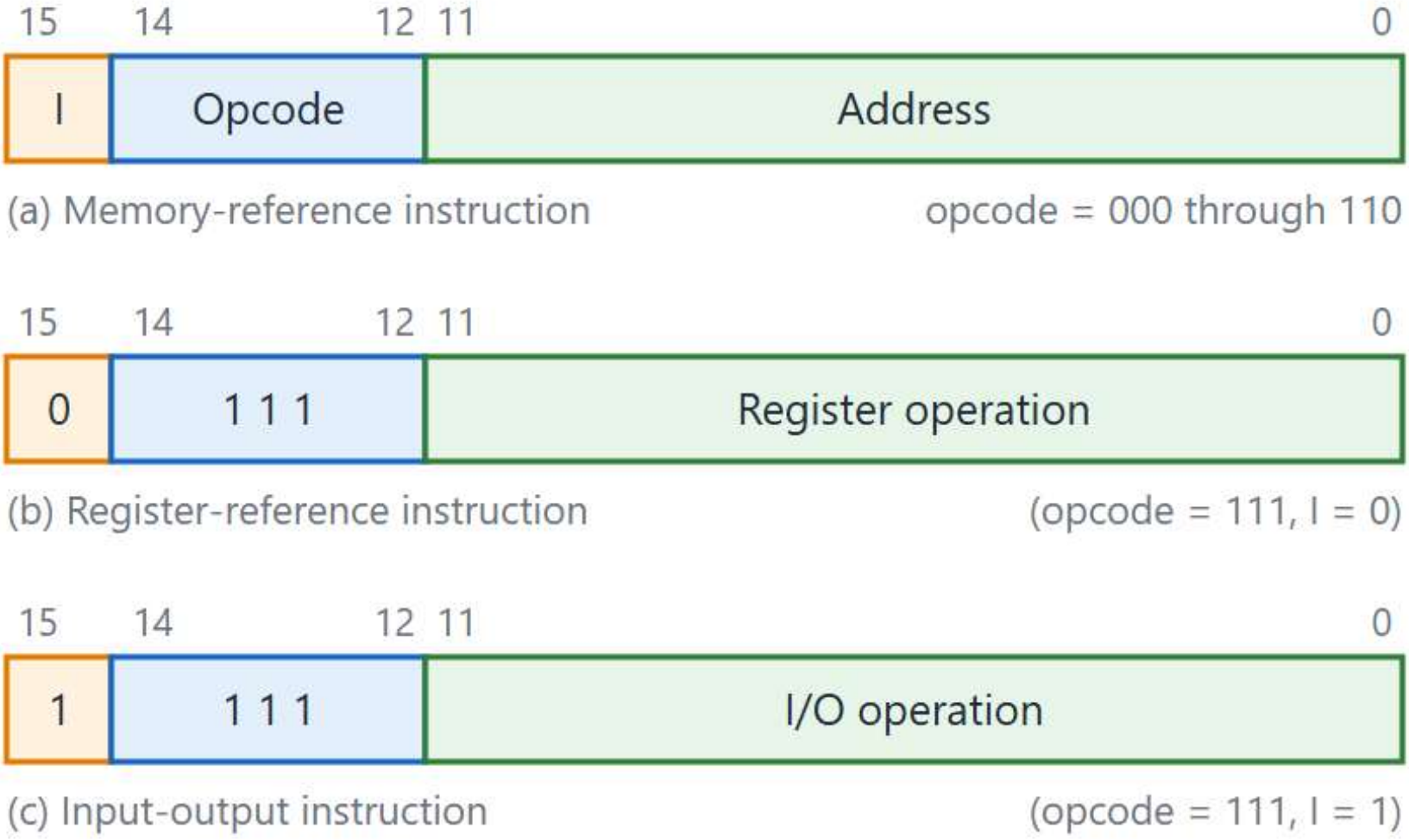
IN SIMPLE TERMS

Opcodes 000–110 are memory instructions with an address; opcode 111 means “no address”. Then the I bit picks register instructions or I/O instructions, and one bit of the low 12 names the exact operation.

How opcode 111 splits in two

- Opcodes 000–110 are memory-reference: the address field points at an operand
- Opcode 111 means the instruction needs no memory address
- Its 12 bits become individual one-hot flags
- The **I** bit (bit 15) picks the decoder: I = 0 register-reference, I = 1 input-output
- 25 instructions in total, in three families
- The control unit tests opcode 111, the I bit and one of bits 11–0

Basic computer instruction formats



Basic computer instruction formats. Adapted from M. Morris Mano, Computer System Architecture, Fig. 5-5.

Memory-reference instructions (7)

Symbol	Opcode	Description
AND	000	AND memory word into AC
ADD	001	Add memory word to AC; carry-out goes to E
LDA	010	Load AC from memory
STA	011	Store AC into memory
BUN	100	Branch unconditionally to the address
BSA	101	Branch and Save return Address (subroutine call)
ISZ	110	Increment memory word; skip next instruction if the result is zero

Register-reference instructions (12)

Opcode 111, I = 0. Each hex code is the full 16-bit word, since there is no separate address.

Symbol	Hex	Description
CLA	7800	Clear AC
CLE	7400	Clear E
CMA	7200	Complement AC
CME	7100	Complement E
CIR	7080	Circular shift right (AC, E)
CIL	7040	Circular shift left (AC, E)

Symbol	Hex	Description
INC	7020	Increment AC
SPA	7010	Skip if AC is positive
SNA	7008	Skip if AC is negative
SZA	7004	Skip if AC is zero
SZE	7002	Skip if E is zero
HLT	7001	Halt: clear the S flip-flop

Input-output instructions (6)

Opcode 111, I = 1.

Symbol	Hex	Description
INP	F800	Input a character into INPR; clear FGI
OUT	F400	Output the character in OUTR; clear FGO
SKI	F200	Skip next instruction if FGI = 1
SKO	F100	Skip next instruction if FGO = 1
ION	F080	Set IEN: interrupts enabled
IOF	F040	Clear IEN: interrupts disabled

3.11 Timing and Control

IN SIMPLE TERMS

The sequence counter ticks T_0, T_1, T_2…; the decoded opcode says which instruction it is. Every control signal is “this instruction AND this tick”.

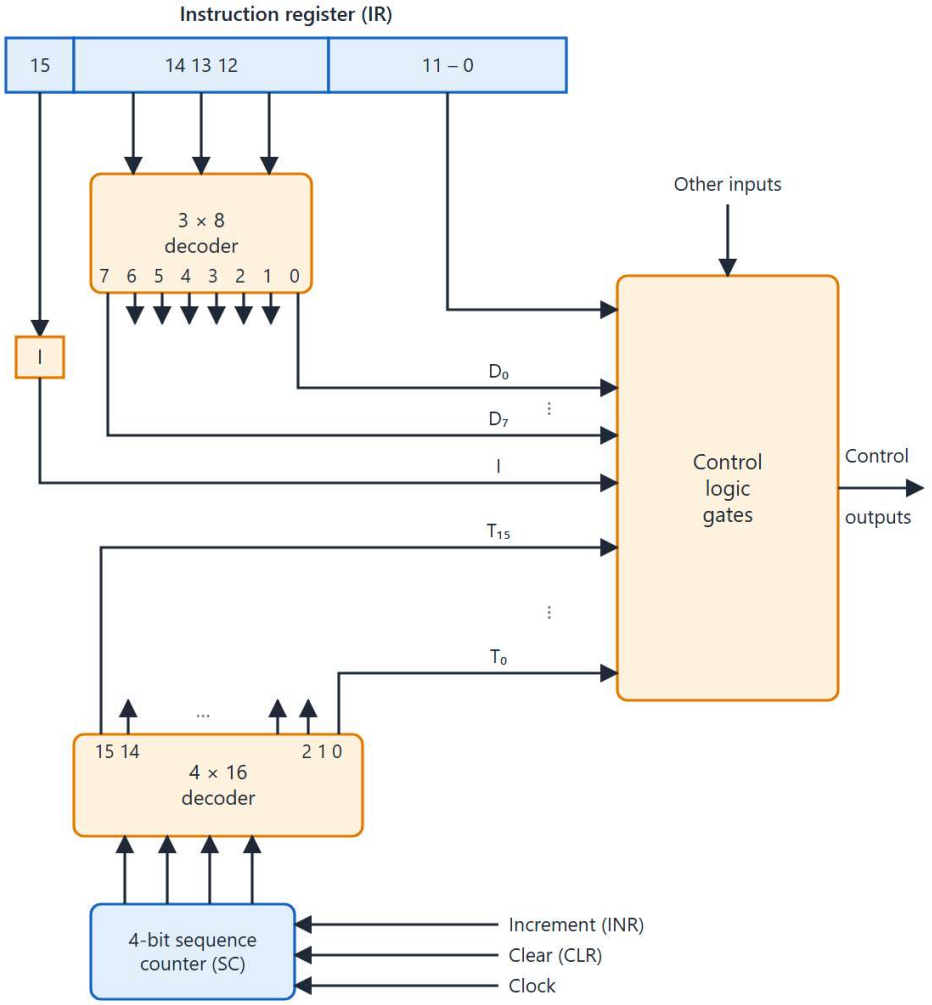
Hardwired control

- Registers hold data and instructions say what to do, but something must decide **when** each register latches
- That is the control unit's job
- The Basic Computer uses a **hardwired** control unit: fixed logic gates, not a program, generate every control signal
- Unit 4's microprogrammed control is the ROM-based alternative

Generating timing signals

- SC, the 4-bit Sequence Counter, feeds a decoder with one active line per state: T_0, T_1, T_2…
- Every clock pulse increments SC, so the active timing signal advances one step per cycle
- Control logic clears SC to 0 to start the next instruction’s fetch
- Most instructions never reach T_6; a simple register-reference instruction can finish by T_3
- SC only counts as far as the current instruction needs

Control unit of the Basic Computer



- The 3 × 8 decoder turns the opcode into D_0–D_7
- The 4 × 16 decoder turns SC into T_0–T_{15}
- Control logic gates combine them with I and IR bits 0–11
- Each control signal = decoder output AND timing signal

Control unit of basic computer. Adapted from M. Morris Mano, Computer System Architecture, Fig. 5-6.

Forming a control statement

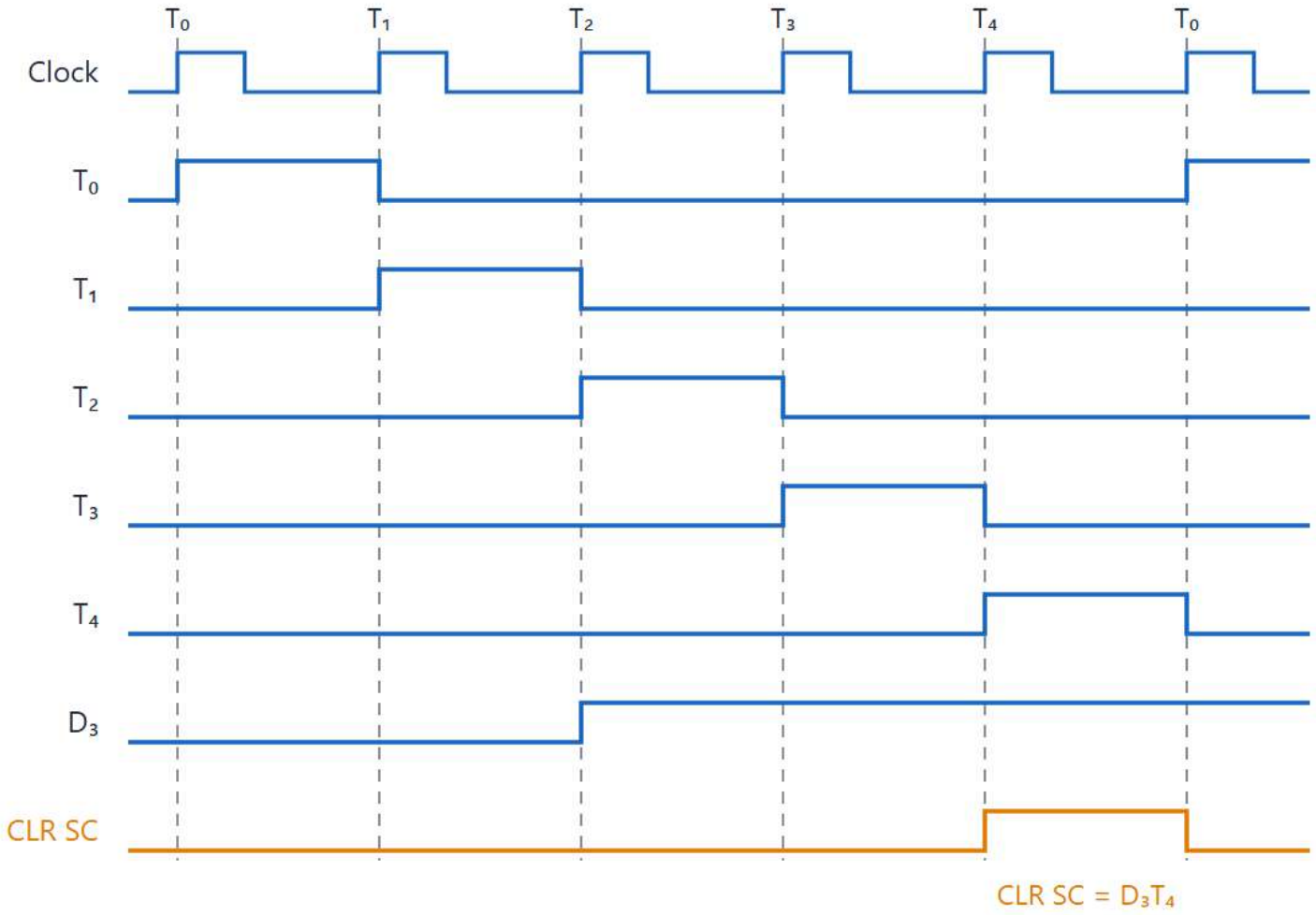
- A timing signal alone does nothing
- It becomes an action when ANDed with an opcode-decoder output and, when needed, a flip-flop condition
- The 3-to-8 decoder reads IR bits 14–12 and produces D_0 through D_7
- Each control signal is the AND of a decoder output and a timing signal
- This is the whole “hardwired” idea in one picture

Example: LDA (D_2)


```
D2T4: DR ← M[AR]
D2T5: AC ← DR, SC ← 0
```

- LDA is opcode 010, so its decoder output is D_2
- It takes two steps, one at T_4 and one at T_5
- The last step clears SC to start the next fetch

Example of control timing signals



Example of control timing signals. Adapted from M. Morris Mano, Computer System Architecture, Fig. 5-7.

3.12 Instruction Cycle

IN SIMPLE TERMS

Every instruction runs the same three opening steps (fetch, decode, find the address) and then follows one of four short execute paths before starting again.

One repeating loop

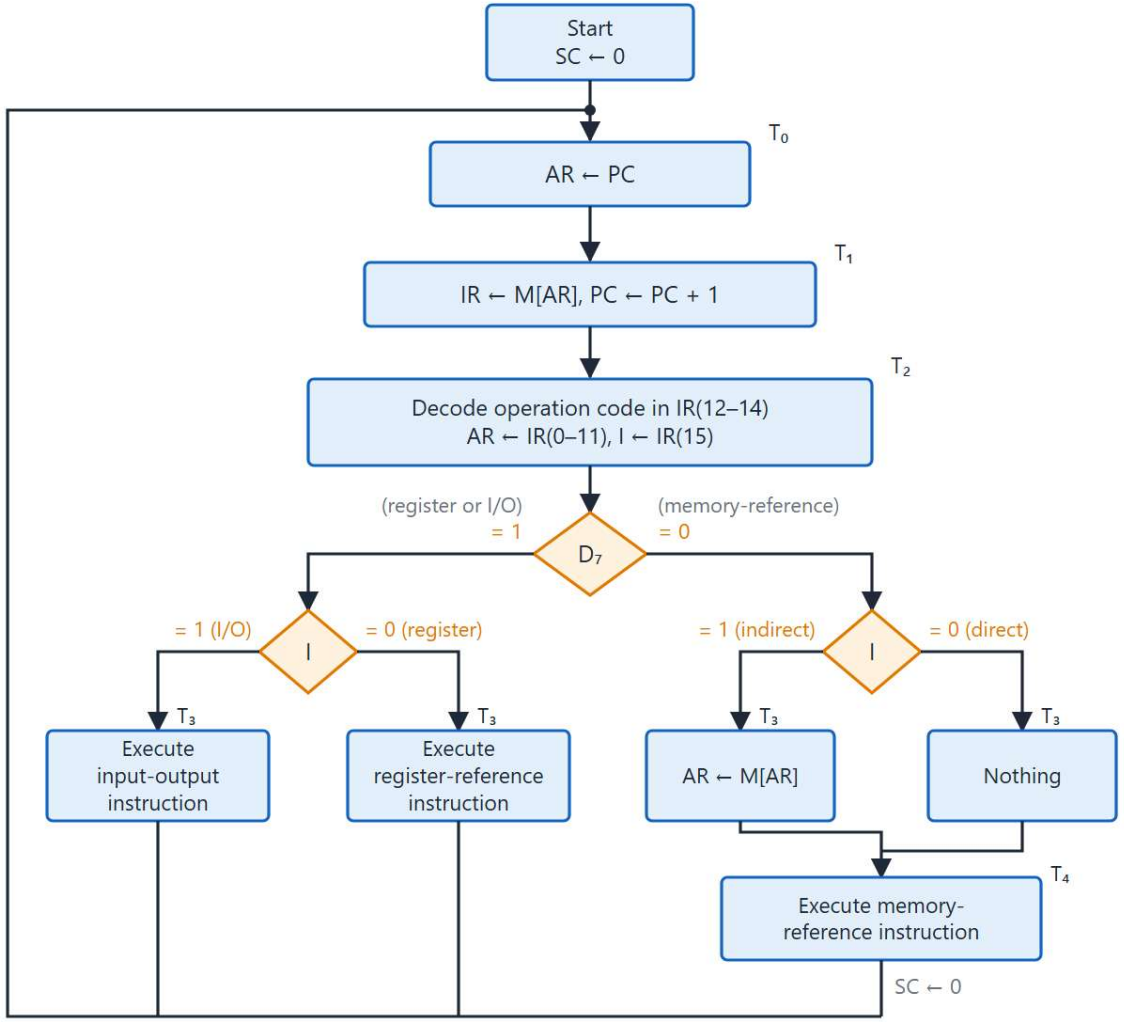
- Everything in this unit exists to run one loop: **fetch** an instruction, **decode** it, **execute** it
- Then do it again for the next instruction
- T_0, T_1 and T_2 are identical for **every** instruction

Fetch and decode: T_0, T_1, T_2


```
T0:   AR ← PC
T1:   IR ← M[AR],   PC ← PC + 1
T2:   D0,…,D7 ← Decode IR(14-12),
         AR ← IR(11-0),   I ← IR(15)
```

- T_0 points AR at the next instruction
- T_1 reads it into IR and advances PC, so PC already points past the current instruction
- T_2 decodes the opcode, copies the low 12 bits into AR as a tentative address, and captures I

Instruction cycle flowchart



- D_7 splits the two kinds of instruction
- $D_7 = 0$: if I = 1, T_3 does AR ← M[AR]; execution starts at T_4
- $D_7 = 1$: I = 0 register-reference, I = 1 input-output, both at T_3
- Every path clears SC; next fetch at T_0

Flowchart for instruction cycle (initial configuration). Adapted from M. Morris Mano, Computer System Architecture, Fig. 5-9.

After decoding at T_2

- **D_7** separates the two kinds of instruction
- $D_7 = 0$ (memory-reference): I = 1 fetches the effective address at T_3 (AR ← M[AR]); I = 0 does nothing
- Execution of a memory-reference instruction then starts at T_4
- $D_7 = 1$: I = 0 is a register-reference instruction, I = 1 is input-output; both execute at T_3
- Every path ends by clearing SC, which starts the next fetch at T_0

3.13 Memory Reference Instructions

IN SIMPLE TERMS

Each memory instruction is a few register transfers, one per clock tick, starting at T_4; most read the memory word into DR first and then use it.

Where execution starts


```
D7' I T3:    AR ← M[AR]
```

- The flowchart compressed all seven instructions into one box; this topic expands it
- Execution always starts at **T_4**, never T_3
- T_3 only resolves an indirect address (when I = 1)
- By T_4, AR holds the operand's real address either way

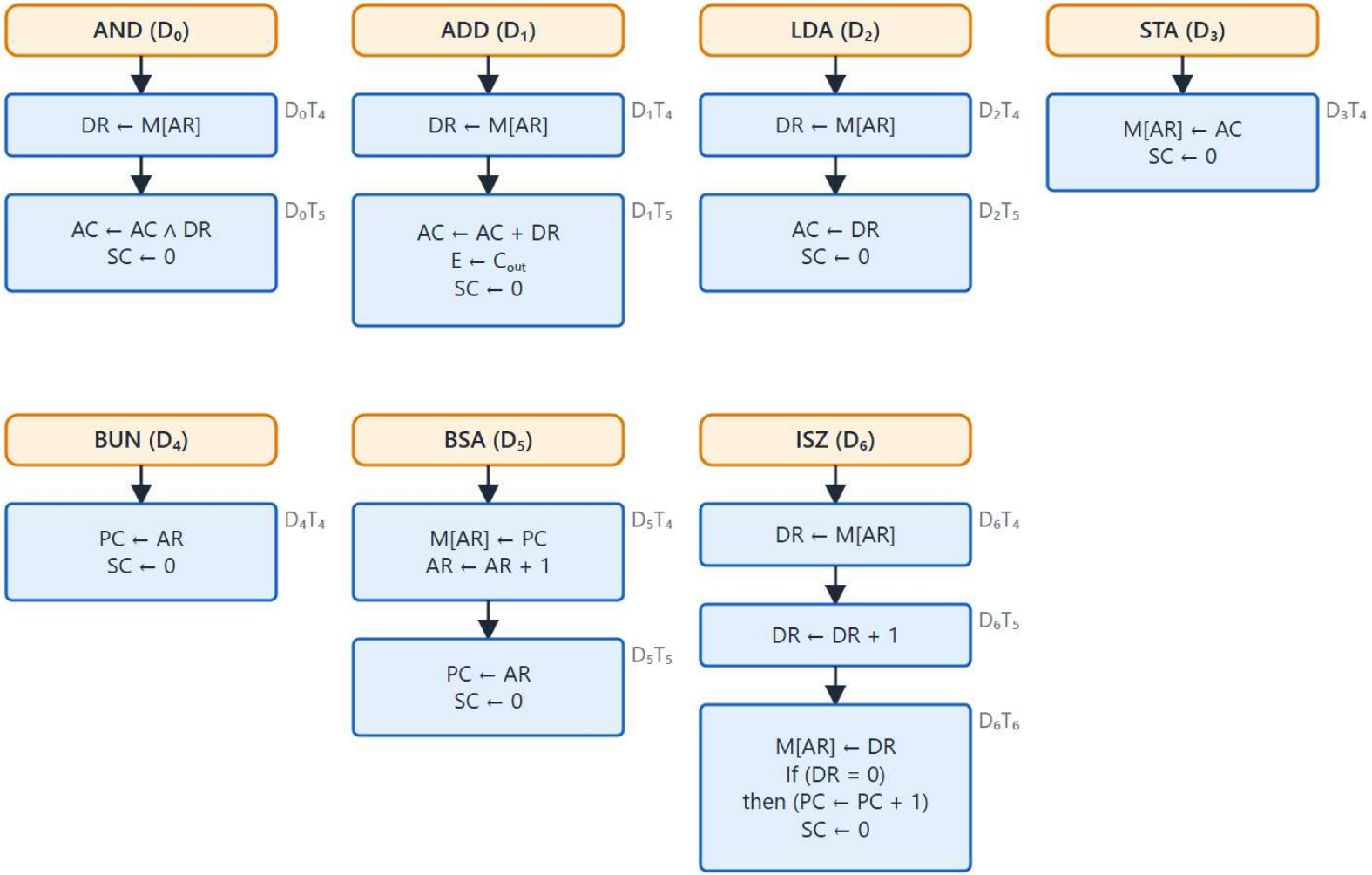
Memory-reference RTL (1 of 2)

Instruction	Timing step and RTL
AND (D_0)	D_0T_4: DR ← M[AR]
	D_0T_5: AC ← AC ∧ DR, SC ← 0
ADD (D_1)	D_1T_4: DR ← M[AR]
	D_1T_5: AC ← AC + DR, E ← C_{out}, SC ← 0
LDA (D_2)	D_2T_4: DR ← M[AR]
	D_2T_5: AC ← DR, SC ← 0
STA (D_3)	D_3T_4: M[AR] ← AC, SC ← 0

Memory-reference RTL (2 of 2)

Instruction	Timing step and RTL
BUN (D_4)	D_4T_4: PC ← AR, SC ← 0
BSA (D_5)	D_5T_4: M[AR] ← PC, AR ← AR + 1
	D_5T_5: PC ← AR, SC ← 0
ISZ (D_6)	D_6T_4: DR ← M[AR]
	D_6T_5: DR ← DR + 1
	D_6T_6: M[AR] ← DR, if (DR = 0) then (PC ← PC + 1), SC ← 0

Memory-reference flowchart



Flowchart for memory-reference instructions. Adapted from M. Morris Mano, Computer System Architecture, Fig. 5-11.

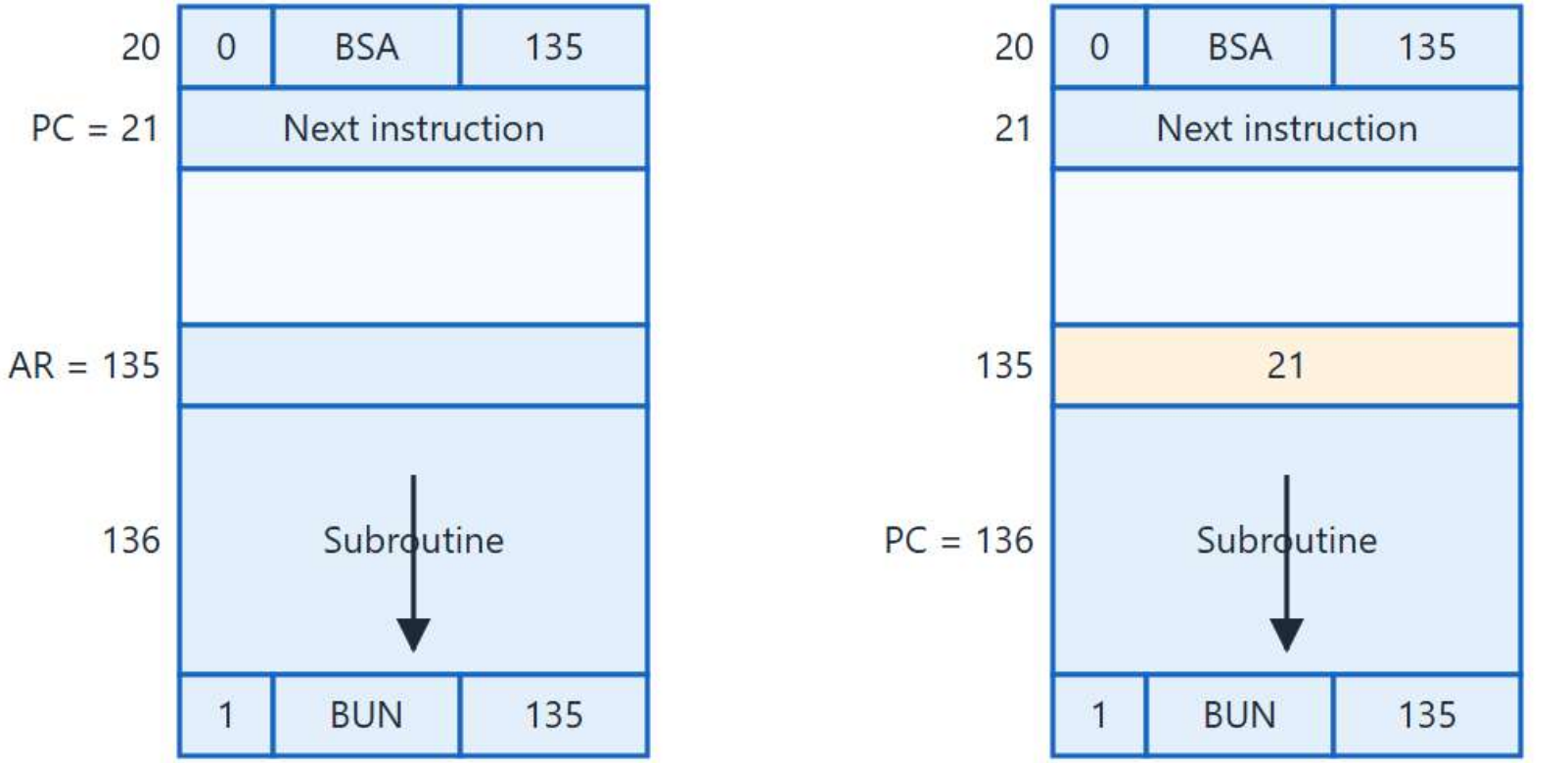
Why DR shows up so often

- AND, ADD, LDA and ISZ all read memory into DR first
- This is a hard constraint: the common bus carries one value per clock pulse
- A memory word must land in a register before it can be combined with AC on the next pulse
- DR exists to be that landing spot

Two instructions worth slowing down on

Instruction	What is actually happening
BSA	Stores the return address (PC, already past this instruction) at the target location, then increments AR and jumps there. The subroutine starts right after its own stored return address, and an indirect BUN through that location returns.
ISZ	Increments the memory word first, then tests whether the new value is zero, not the original. A built-in loop counter that counts up to zero.

BSA instruction example



Example of BSA instruction execution. Adapted from M. Morris Mano, Computer System Architecture, Fig. 5-10.

3.14 Input-Output and Interrupt

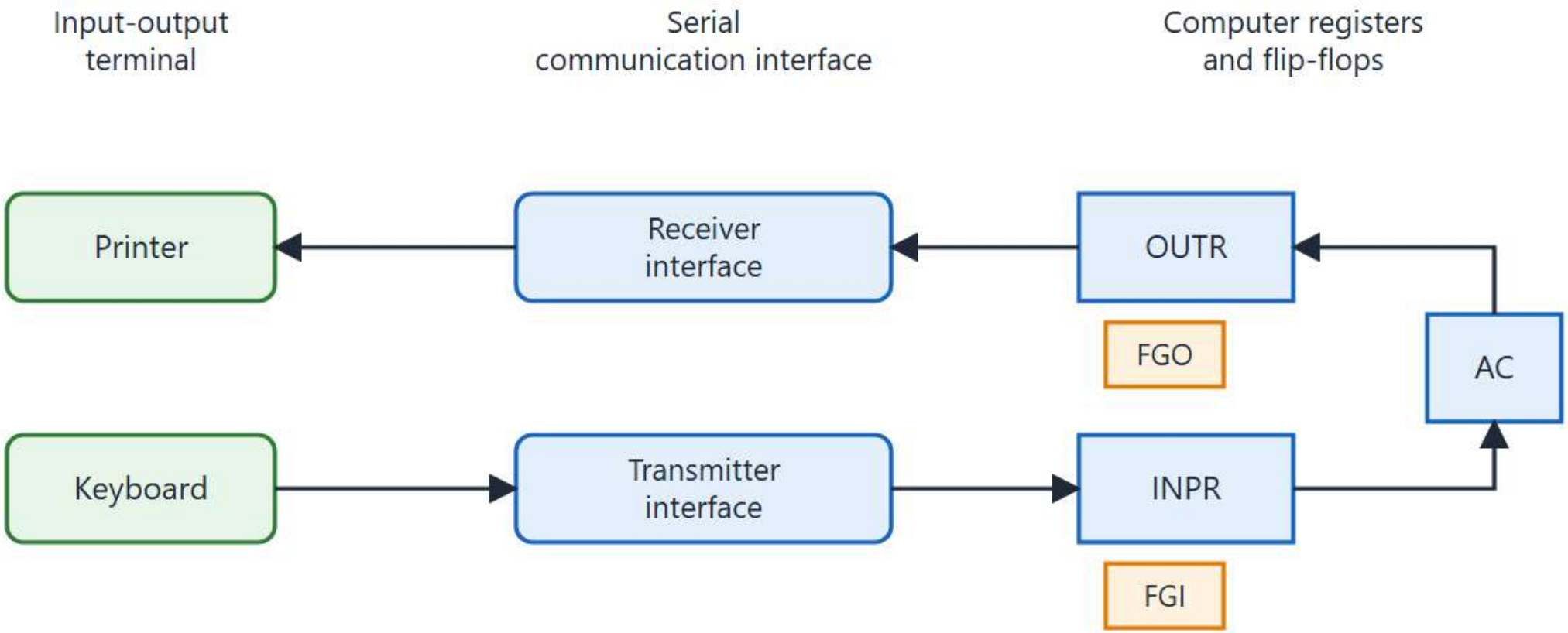
IN SIMPLE TERMS

Devices raise a flag when ready. With interrupts enabled, the computer finishes the current instruction, saves where it was, runs the device's service program and then comes back.

The I/O model

- The picture is a terminal: a keyboard feeding INPR and a printer fed by OUTR
- Each has a ready flag that the **device itself** sets
- **FGI** = 1: the input device has put a character in INPR
- **FGO** = 1: the output device has finished and can take another character
- The CPU only **clears** these flags; it never sets them

Input-output configuration



Input-output configuration. Adapted from M. Morris Mano, Computer System Architecture, Fig. 5-12.

RTL for the I/O instructions


```
p = D7 I T3    (opcode 111, I = 1, timing T3)

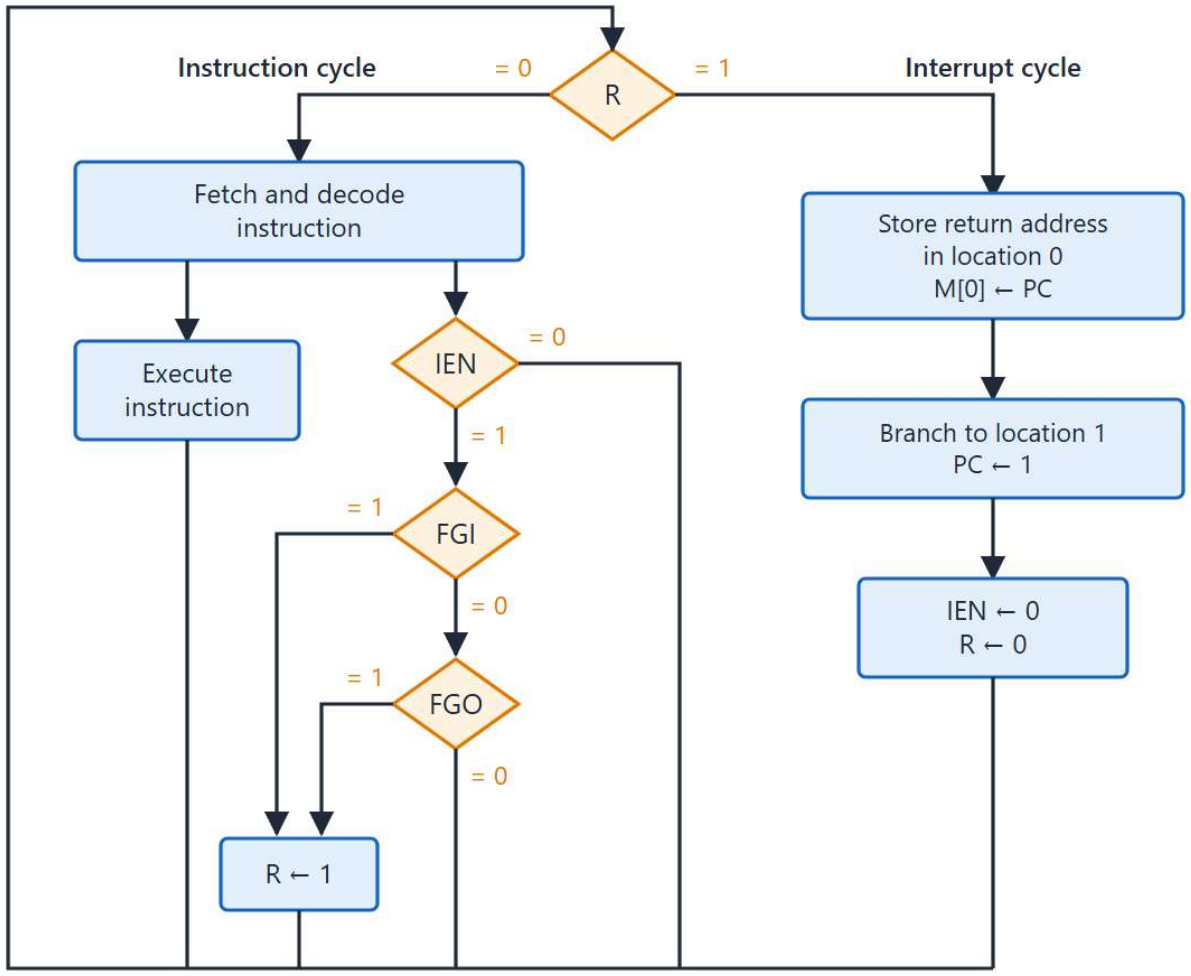
pB11 (INP):  AC(0-7) ← INPR,  FGI ← 0
pB10 (OUT):  OUTR ← AC(0-7),  FGO ← 0
pB9  (SKI):  if (FGI = 1) then (PC ← PC + 1)
pB8  (SKO):  if (FGO = 1) then (PC ← PC + 1)
pB7  (ION):  IEN ← 1
pB6  (IOF):  IEN ← 0
all:  SC ← 0
```

- All six execute in one state, T_3: no indirect-address detour

From polling to interrupts

- SKI and SKO let a program **poll**: loop on SKI until it skips, then INP
- Polling is exactly the problem interrupts exist to solve
- IEN (interrupt enable) and R (interrupt flip-flop) are checked at the boundary between instructions
- That is where SC would normally reset to 0 for the next fetch

Interrupt cycle flowchart



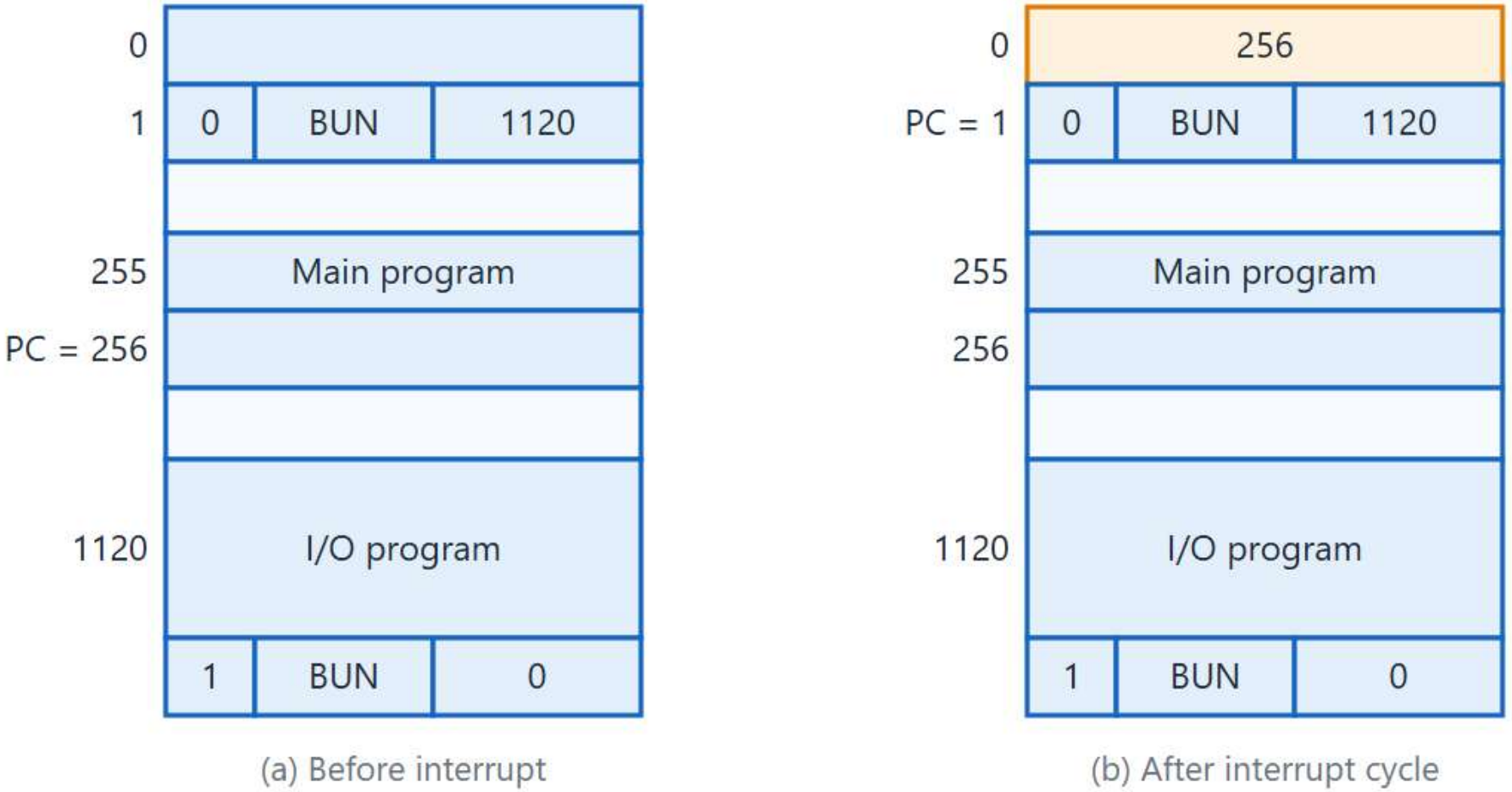
- IEN and R are checked between instructions
- Interrupt cycle saves PC at location 0 and sets PC to 1
- Location 1 holds a BUN to the service routine
- IEN and R are then cleared

Flowchart for interrupt cycle. Adapted from M. Morris Mano, Computer System Architecture, Fig. 5-13.

When the interrupt is granted

- The interrupt cycle saves the return address (the interrupted program's PC) at fixed memory location 0
- It then sets PC to 1
- Location 1 is expected to hold a BUN to the interrupt-service routine
- Execution resumes there, with the way back already stored

Demonstration of the interrupt cycle



Demonstration of the interrupt cycle. Adapted from M. Morris Mano, Computer System Architecture, Fig. 5-14.

Unit 3 summary

- **RTL** writes register transfers as R2 ← R1; a control condition before a colon makes a transfer conditional
- A shared **bus** (multiplexers or three-state buffers) lets many registers use one set of lines; memory uses AR, DR and M[AR]
- **Arithmetic, logic and shift** micro-operations combine into one **ALU**, chosen by a select code
- The **Basic Computer** has 16-bit instructions (I, opcode, address) and registers on one common bus
- The hardwired **control unit**: SC and decoders give T and D signals; each control signal is D AND T
- The **instruction cycle** is fetch, decode, execute; I/O flags and the **interrupt** let devices get the CPU's attention

Unit 3: key applications

- **3.1–3.3:** RTL appears in datapath documents and in Verilog and VHDL; every register load uses a load-enable; CPU and system buses share wires with MUXes or three-state buffers
- **3.4–3.7:** add, subtract and compare go through one adder; PC ← PC + 1 is an increment; masks extract and set bit fields; shifts multiply, divide and mix bits; ALUs such as the 74181 select among all of these
- **3.8–3.10:** real instruction sets use a fixed opcode field; indirect addressing is the ancestor of pointers; BSA is the ancestor of CALL
- **3.11–3.12:** hardwired control (counter, decoder, AND gates); fetch, decode, execute is the baseline that pipelining overlaps
- **3.13–3.14:** DR staging and ISZ loop counters; interrupt vector tables and device ready flags